

Experiment # 05

Implementation of Image Histogram and Equalization

Objective

1. To implement image histogram
2. To implement histogram equalization

Software

- Python Idle 3.7 or 3.8.5
- Spyder

Theory

Histogram Calculation

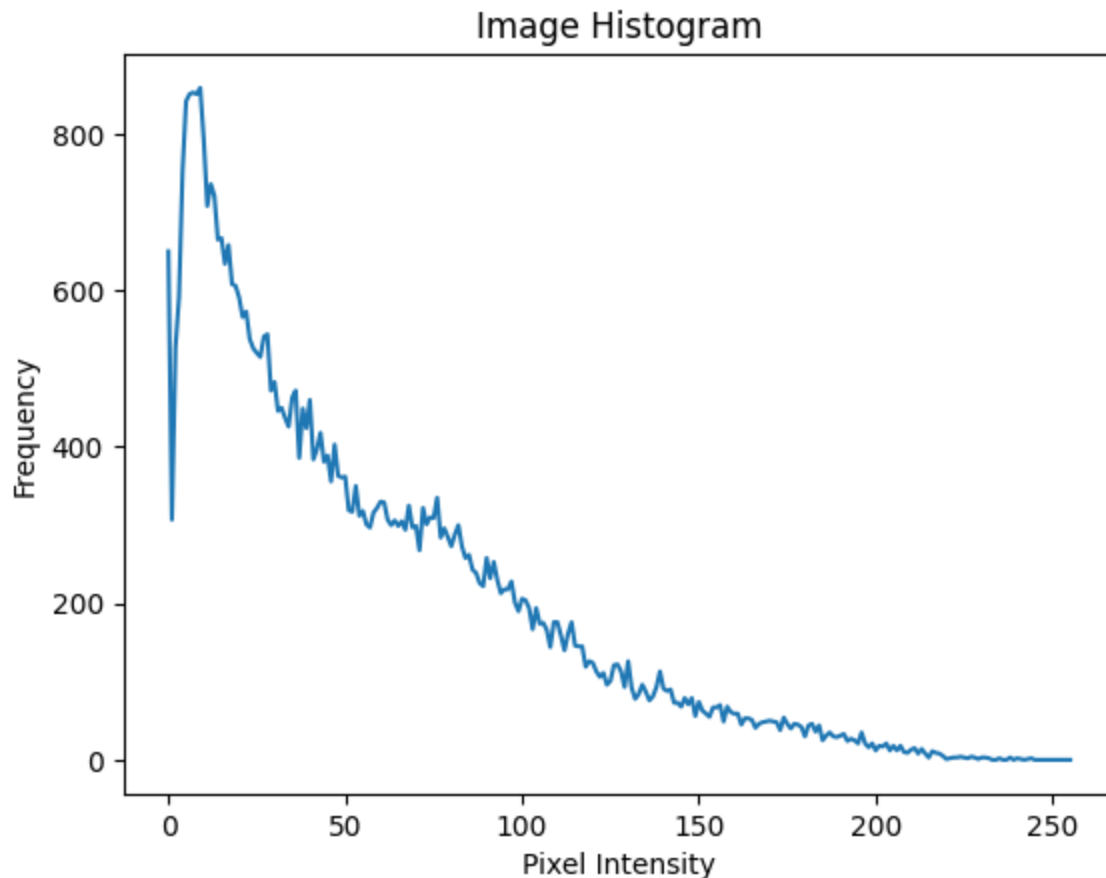
Python Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load the image in grayscale
image = cv2.imread('download.jpeg', 0)

# Calculate the histogram using numpy.histogram()
# 'bins=256' means we want 256 intensity levels (for grayscale images, 0 to 255)
# 'range=(0, 256)' means we are considering pixel intensities in this range
hist, bin_edges = np.histogram(image, bins=256, range=(0, 256))
#histogram = cv2.calcHist([image], [0], None, [256], [0, 256])

# Plot the histogram
plt.plot(bin_edges[0:-1], hist)
plt.title('Image Histogram')
plt.xlabel('Pixel Intensity')
plt.ylabel('Frequency')
plt.show()
```



Histogram equalization:

Histogram of an image shows the frequency of different intensities values present in the image. This gives a clear idea of what intensities dominate the image. **Histogram equalization** is a technique that uses this information to enhance the contrast using the probability of a certain pixel to occur. The Image obtained will be Histogram Equalized Image with high contrast.

Python Code:

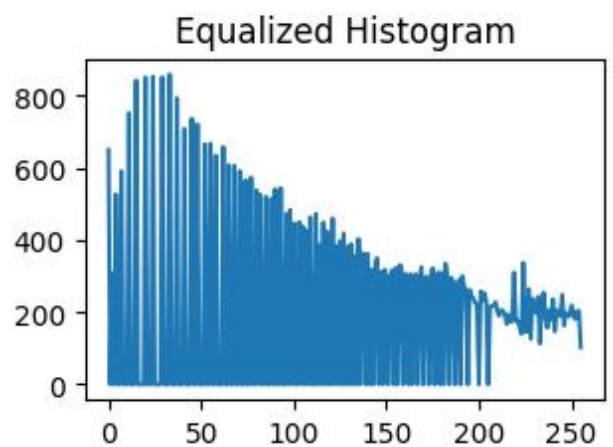
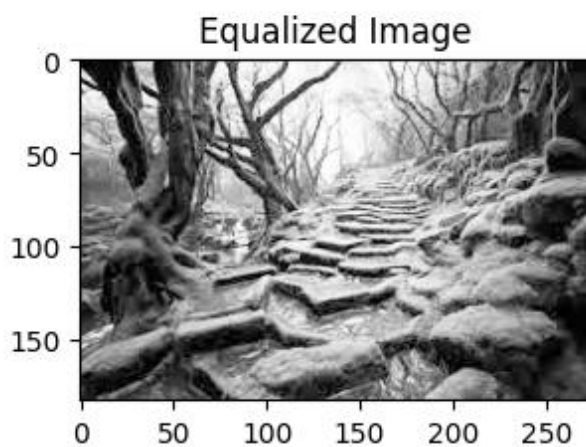
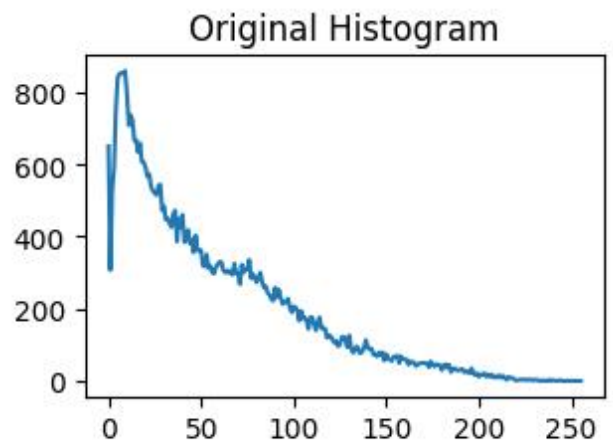
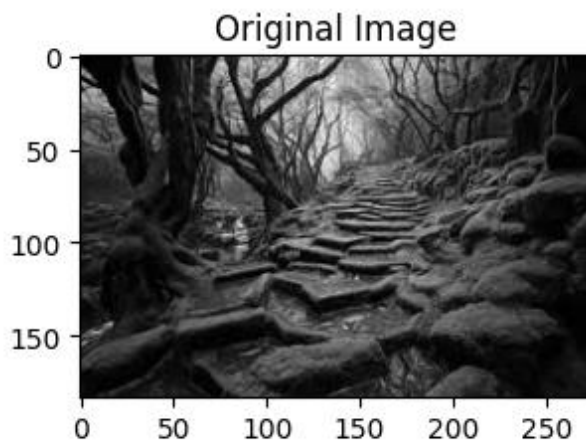
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Load the image in grayscale
image = cv2.imread('download.jpeg', 0)
# Perform Histogram Equalization
equalized_image = cv2.equalizeHist(image)
# Calculate the histogram of the equalized image
equalized_histogram = cv2.calcHist([equalized_image], [0], None, [256], [0, 256])
# Display the original and equalized image side by side
plt.subplot(2, 2, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
```

```
plt.subplot(2, 2, 2)
plt.plot(histogram)
plt.title('Original Histogram')

plt.subplot(2, 2, 3)
plt.imshow(equalized_image, cmap='gray')
plt.title('Equalized Image')

plt.subplot(2, 2, 4)
plt.plot(equalized_histogram)
plt.title('Equalized Histogram')

plt.show()
```



Some Useful Commands:

Importing matplotlib:

```
import matplotlib.pyplot as plt
```

1. To plot a simple plot using matplotlib:

```
plt.plot( my_data)
```

2. For histogram calculation:

```
numpy.histogram(img, bins, range=None, normed=None, weights=None, density=None)
```

3. For histogram equalization:

```
cv2.equalizeHist(img)
```

4. For label along x axis:

```
plt.xlabel ( 'Some cooked up data')
```

5. For label along y axis:

```
plt.ylabel ( 'Some value')
```

6. To show the graph:

```
plt.show()
```

Lab Tasks:

1. Apply Histogram equalization on the given image by applying above steps.



- i. Calculate the histogram of the image save it as Figure_1.jpg.
- ii. Calculate probability density function (PDF) from the histogram and save it as Figure_2.jpg. $PDF = H/(R \times C)$. Where H is the Histogram and R and C is the number of Rows and Columns of the image respectively.
- iii. Calculate cumulative density function (CDF) and save it as Figure_3.jpg.
- iv. Multiply the Cumulative PDF with 255 to find the transformation function then save it as Figure_4.jpg

- v. From the transformation function, replace the gray levels of the image to create contrast enhanced (histogram equalized) image and save it as Figure_5.jpg
 - vi. Calculate the histogram of the output image save it as Figure_6.jpg.
2. For the given image
- i. Calculate histogram of the given image using build-in functions of numpy or OpenCV.
 - ii. Calculate histogram equalization of the given image using build-in functions of numpy or OpenCV and display your output.